Université Paris 1 Panthéon-Sorbonne

Master 2 TIDE — Traitement de l'Information et Data Science en Entreprise

Apprentissage Statistique

Analyse des annulations de réservations hôtelières

INN Hotels Group — Portugal

Étudiants

HADMEN Moad
KESSI Brahim
MATTEI Alexis

Professeur

CELISSE Alain

Groupe 2 — Année universitaire 2025–2026 — 9 mars 2026

Table des matières

Introduction . . . 2
Contributions . . . 2
Présentation des données et preprocessing commun . . . 2
1 **EDA** . . . 3
2 **Featuring** . . . 3
3 **Modèle de prédiction** . . . 4
3.1 Régression logistique . . . 4
3.2 Approche générative . . . 4
3.2.1 LDA : Linear Discriminant Analysis . . . 4
3.2.2 QDA : Quadratic Discriminant Analysis . . . 5
3.3 Approche régularisée (Lasso + Ridge) . . . 5
3.3.1 Rappel formule Lasso Ridge . . . 5
3.3.2 Lasso + GridSearchCV . . . 6
3.3.3 Ridge + GridSearchCV . . . 6
3.3.4 Threshold Tuning via Lasso . . . 6
3.4 Approche Géométrique . . . 7
3.5 Performance pure : arbre de décision . . . 8
3.5.1 Random Forest . . . 8
3.5.2 XG BOOST . . . 8
3.5.3 Synthèse comparative . . . 9
3.5.4 Analyse business : impact financier . . . 10
4 **Q3 — Identification de profils de réservations par clustering GMM** . . . 11
4.1 Question et motivation . . . 11
4.2 Méthodologie . . . 11
4.3 Résultats . . . 12
4.4 Conclusion . . . 13
A **Annexe- Modèlisation** . . . 14
A.1 LDA et QDA . . . 14
A.1.1 LDA . . . 14
A.1.2 QDA . . . 14
A.2 KNN . . . 15
A.3 XG BOOST . . . 16
Annexe — EDA . . . 16

Introduction

Contributions

- **Moad Hadmen** (Q1) : EDA, feature engineering, régression logistique (base + Lasso/Ridge + threshold tuning), Random Forest, synthèse comparative et analyse business (simulation financière, segmentation des profils à risque, recommandations opérationnelles).
- **Brahim Kessi** (Q2) : Approche générative (LDA, QDA : modéliser la distribution des données), approche géométrique (KNN : vérification de la compléxité local des variables (chevauchement), XGBoost(determination du meilleur modèle et analyse comparative avec le Random Forest).
- **Alexis Mattei** (Q3) : Identification de profils homogènes de réservations par clustering non supervisé (GMM), avec réduction dimensionnelle préalable par ACP, sélection du nombre de clusters via BIC/AIC et interprétations des quatre profils identifiés en termes de risque d'annulation.

Présentation des données et preprocessing commun

Le jeu de données INN Hotels Group contient **36 275 réservations** décrites par 19 variables (caractéristiques du séjour, profil client, informations tarifaires et statut d'annulation). Aucune valeur manquante n'est présente, ce qui constitue une base solide pour la modélisation. La variable cible `booking_status` indique si la réservation a été annulée (33%) ou honorée (67%).

En ce qui concerne le nettoyage des données, on a supprimé les identifiants (`Booking_ID`), réalisé un encodage *one-hot* des trois variables catégorielles (`type_of_meal_plan`, `room_type_reserved`, `market_segment_type`) et effectué une standardisation Z-score de l'ensemble des variables. Ce preprocessing produit une matrice de **27 variables** utilisée comme base commune pour toutes les analyses.

1. EDA

Moad

Le jeu de données contient 36 275 réservations et 19 variables, sans aucune valeur manquante. La cible est déséquilibrée : 67 % de réservations honorées contre 33 % d'annulations, ce qui oriente nos choix de métriques vers le Recall.

On a réalisé des boxplots qui croisent la variable target avec les variables numériques.

— Le `lead_time` ressort comme la variable la plus discriminante ($r = 0{,}44$ avec la cible) : il semble que les clients qui réservent longtemps à l'avance annulent bien plus souvent que les autres clients.

— Le `avg_price_per_room` ($r = 0{,}14$) montre aussi un écart entre les deux classes.

Le test de Mann-Whitney confirme ces différences ($p < 0{,}001$ pour les deux variables).

Au niveau des variables catégorielles, le test du Chi2 d'indépendance montre que `market_segment_type` ($\chi^2 = 814$, $p < 10^{-175}$), `type_of_meal_plan` ($\chi^2 = 278$, $p < 10^{-60}$) et `room_type_reserved` ($\chi^2 = 57$, $p < 10^{-10}$) sont toutes associées à la target.

La matrice de corrélation montre une corrélation négative de `no_of_special_requests` avec la cible ($r = -0{,}25$) : il semble que plus un client personnalise son séjour moins il annule, ce qui semble logique d'un point de vue métier. Le jour du mois (`arrival_date`) n'apporte rien ($r = 0{,}01$) et sera supprimé.

L'analyse temporelle par mois d'arrivée (figure 1) montre une légère saisonnalité : le taux d'annulation varie selon les mois avec des pics sur certaines périodes. Sinon l'ensemble des graphiques de l'EDA (répartition de la cible, boxplots, distributions, variables catégorielles, matrice de corrélation) est disponible en Annexe A.3.

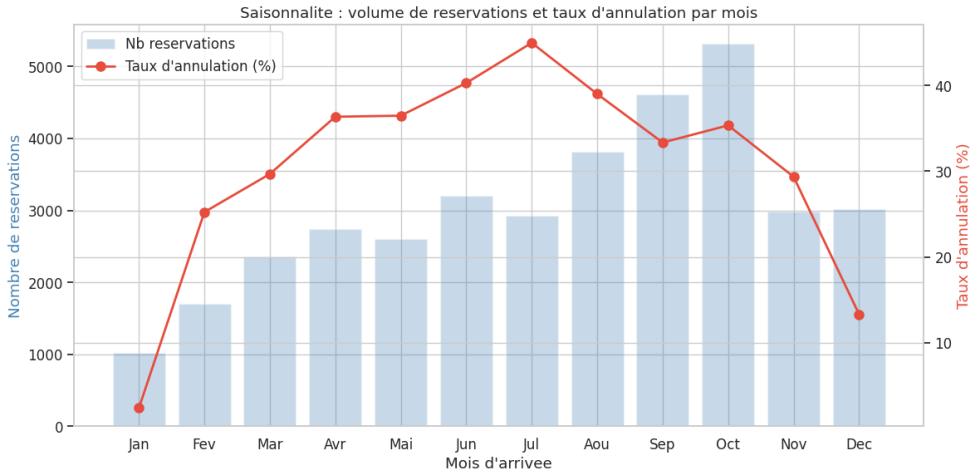

FIGURE 1 – Saisonnalité : volume de réservations et taux d'annulation par mois d'arrivée.

2. Featuring

Moad

La colonne `arrival_date` est supprimée ($r = 0{,}01$).

On construit 13 features métier : `total_nights` (durée du séjour), `total_guests` (taille du groupe), `is_long_lead` (réservation > 90 jours), `has_children`, `arrival_quarter`, `estimated_total_price`, `historical_cancellation_rate`, `is_new_guest`, `weekend_ratio`, `has_special_requests`, `price_per_person`, `is_free_room` et `is_weekend_only`.

Ces variables dérivées peuvent introduire de la colinéarité, mais le Lasso se chargera d'éliminer les redondances pour les modèles linéaires, et les modèles ensemblistes n'y sont pas sensibles.

— Avec l'encodage one-hot des 3 variables catégorielles (avec `drop_first`) on obtient une matrice de **39 features**.

— Le split train/test est stratifié (80/20, soit 29 020 / 7 255 observations) et une standardisation Z-score est appliquée.

3. Modèle de prédiction

3.1 Régression logistique

Moad

On commence par une régression logistique simple avec `class_weight='balanced'`.

Performances sur la classe *Canceled* :

— **Recall :** 0,79
— **Precision :** 0,65
— **F1-Score :** 0,71

Métriques globales :

— **AUC-ROC :** 0,8722
— **AUC-PR :** 0,7760

Validation croisée (5-fold) : La cross-validation confirme la stabilité du modèle :

— **Recall CV :** $0,776 \pm 0,007$
— **AUC-ROC CV :** $0,863 \pm 0,005$

Dans notre contexte métier, un faux négatif (c'est-à-dire une chambre vide non anticipée) coûte bien plus cher qu'un faux positif (surbooking maîtrisé). Un **Recall de 0,79** est donc un bon point de départ, et on dispose maintenant d'une baseline interprétable et robuste pour la suite.

3.2 Approche générative

BRAHIM

La Régression Logistique est un modèle discriminant : elle cherche directement à modéliser $P(Y|X)$. Pour approfondir notre démarche scientifique, nous allons maintenant tester des modèles génératifs, qui modélisent la distribution des données : la LDA et la QDA.

Dans la partie EDA (voir la figure 22), nous avons conclu à la non-normalité de certaines de nos variables. Or cette condition est nécessaire dans ces modèles, cela va nous permettre de vérifier si ces modèles restent performants malgré les écarts à la normalité constatés.

3.2 LDA : Linear Discriminant Analysis

résultat :

voir la figure 12.

Notre AUC -ROC (0.8683) est très proche de celui que nous avons obtenu avec la régression simple (0.8724), cela implique que la frontière qui sépare les deux classes que nous étudions est linéaire.

Toutefois le recall de canceled (annulation) est largement plus faible que celui précédemment obtenu ($0.62 < 0.79$). Cela est dû au fait que la LDA rate plus d'annulations mais elle fait moins de fausses alertes.

En terme de structure de données, cela s'explique par le fait que la LDA a plus de mal à trancher dans les zones de chevauchement (overlap).

Globalement, malgré que les variables ne sont pas parfaitement normales, le modèle obtenu est assez robuste (AUC-ROC = 0.86).

Pour essayer de régler le problème du recall, nous allons tenter l'approche via le QDA (Quadratic Discriminant Analysis).

3.2 QDA : Quadratic Discriminant Analysis

Le QDA permet de tracer des coubres (plus souple que la LDA qui elle est lineaire). Grace à cela , nous alllons éssayer d'améliorer la prise de désicion au niveau des zones de chevauchements

Résultat :

voir la figure 13.

Dans un premier temps, nous avons essayé avec le reg_param = 0.1 et nous avons obtenu des résultats médiocres sur l'Accuracy (0.47) alors que l'AUC-ROC (0.8395) est en fait très correct.

Cela signifie que le modèle arrive à définir une hiérarchie avec les probabilités des clients mais le seuil de décision est très mauvais, il prédit "Canceled" pour presque tout le jeu de données.

Cela peut être dû à l'existence d'une multicolinéarité suite à l'ajout de nouvelles variables. Ou c'est la non-normalité des variables qui se fait ressentir, puisque le QDA est plus sensible que la LDA.

Avec un reg_param = 0.8, le modèle devient plus rigide, il ignore les corrélations entre les variables. On augmente le biais (modèle plus simple) mais on réduit la variance.

Malheureusement l'AUC-PR est largement meilleure pour la LDA (0.6916 (QDA) « 0.7714 (LDA)). Or cette métrique est très importante puisque elle permet de savoir si le modèle arrive à isoler les annulations (ce que l'on cherche à faire).

Avec le reg_param = 0.1, on obtient une meilleure AUC-PR (0.71) mais cela est faussé par le fait que le modèle prédit l'annulation pour presque tout le monde.

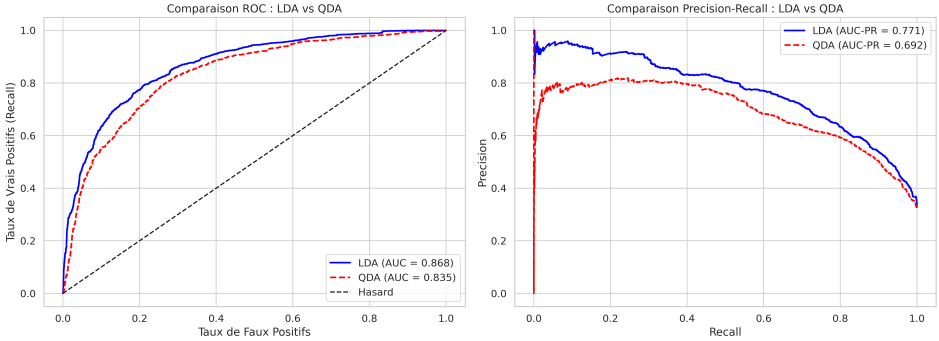

FIGURE 2 – LDA vs QDA

A ce stade, nous pouvons déjà nous avancer sur la forme de la frontière idéale pour pouvoir prédire les annulations qui serait plus linéaire (de meilleures performances sur le LDA comme nous le voyons sur le graphique ci-dessus). Mais nous avons aussi observé que la complexité des données introduit une forte instabilité dans l'estimation des paramètres comme nous l'avons vu avec la QDA.

3.3 Approche régularisée (Lasso + Ridge)

Moad

Puisque la frontière de décision est linéaire, nous utilisons une régression logistique régularisée pour améliorer nos premiers modèles.

3.3 Rappel formule Lasso Ridge

Moad

- **La pénalité L1 (Lasso)** pénalise en valeur absolue et force des coefficients exactement à zéro : c'est une sélection automatique de variables.
- **La pénalité L2 (Ridge)** réduit les coefficients de manière quadratique sans jamais les annuler, et gère la multicolinéarité en répartissant les poids entre features corrélées.
- **L'intérêt métier concret du Lasso** est de dire à l'hôtel : au lieu de surveiller 39 indicateurs,

voici les variables qui comptent vraiment.

3.3 Lasso + GridSearchCV

Moad

Le GridSearchCV (scoring Recall, 5 folds) teste $C \in \{10^{-4}, \ldots, 100\}$ et identifie $C = 0{,}001$ comme optimal (Recall CV = 0,823 ± 0,009).

À cette régularisation, le Lasso ne conserve que **8 variables sur 39** :

- `lead_time` (coeff. = 0,84),
- `no_of_special_requests` (−0,48),
- `market_segment_type_Online` (0,38),
- `avg_price_per_room` (0,18),
- `has_special_requests` (−0,13),
- `arrival_year` (0,09),
- `estimated_total_price` (0,06) et
- `is_long_lead` (0,02).

Les 31 variables éliminées n'apportaient que du bruit.

Sur le test : Recall = 0,82, Precision = 0,62, AUC-ROC = 0,8623.

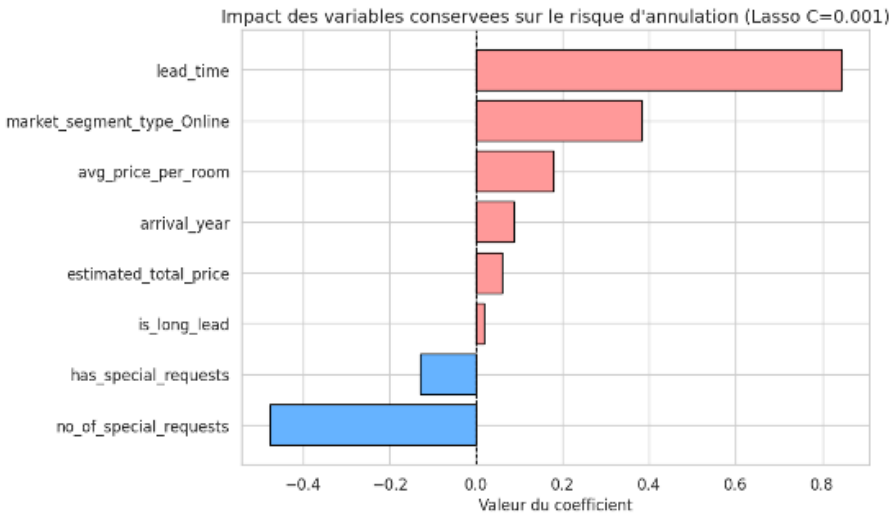

FIGURE 3 – Coefficients des variables conservées par le Lasso ($C = 0{,}001$). En rouge : augmente le risque d'annulation. En bleu : le réduit.

3.3 Ridge + GridSearchCV

Moad

On effectue la même démarche avec cette fois une pénalité L2. On fixe le Ridge avec $C = 0{,}0005$ (Recall CV = 0,801 ± 0,012), ce qui conserve toutes les variables mais redistribue les poids entre features corrélées. Les performances sont proches du Lasso : Recall = 0,80, AUC-ROC = 0,8653. La frontière est confirmée comme robuste.

3.3 Threshold Tuning via Lasso

Moad

En fixant un objectif de **Recall = 0,85**, le seuil optimal est identifié à **0,48** (au lieu de 0,50). La Precision descend à 0,60, ce qui est acceptable car vaut mieux vaut quelques fausses alertes supplémentaires que des chambres vides non anticipées.

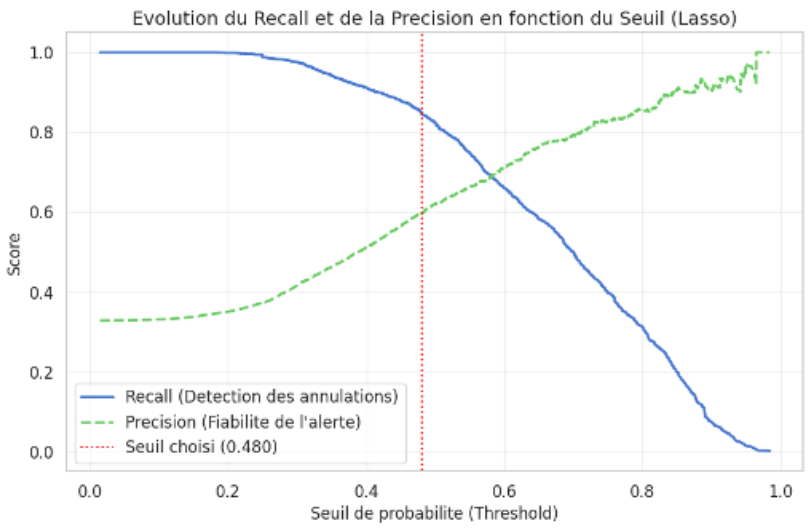

FIGURE 4 – Évolution du Recall et de la Precision en fonction du seuil (Lasso).

3.4 Approche Géométrique

Brahim

Nous avons donc à ce stade confirmé la linéarité de notre frontière (de bons résultats avec le lasso/ridge).

Cette optimisation de la régression logistique nous a permis de stabiliser nos résultats avec l'approche linéaire globale tout en sélectionnant les variables les plus discriminantes.

Mais tous les modèles que l'on a vus jusqu'à maintenant imposent une règle de décision rigide à l'ensemble du nuage de points. Pour clore notre analyse des modèles classiques du cours, nous introduisons le k-NN (k-Nearest Neighbors).

Cette approche non paramétrique et locale ne définit pas une loi globale sur nos données mais se penche sur les structures et complexités locales de nos données.

Cela va nous permettre de voir si on peut ajouter de l'information supérieure à la tendance globale décelée par nos modèles en regardant les k plus proches voisins.

En lançant le GridSearchCV , on obtient que le meilleur k est 11. (voir la figure 14)

Résultat :

voir la figure 5.

Nous pouvons être très content des résultats. En effet, le KNN a réussi là où la LDA a échoué. Je m'explique, avec de tels résultats, cela signifie que la frontière n'est pas seulement linéaire mais qu'il existe à certains endroits une certaine complexité (un chevauchement des clients), là où la LDA ne réussissait pas à bien prédire l'étiquette d'un client.

Quand le modèle (KNN) prédit une annulation, il a raison dans 80 % des cas (Précision). C'est beaucoup plus fiable que tous les modèles précédents. De plus, il attrape 73 % des annulations réelles (Recall). De même que l'AUC-PR (0.8393) où on peut voir que le modèle arrive très bien à prédire si un client va annuler sa réservation.

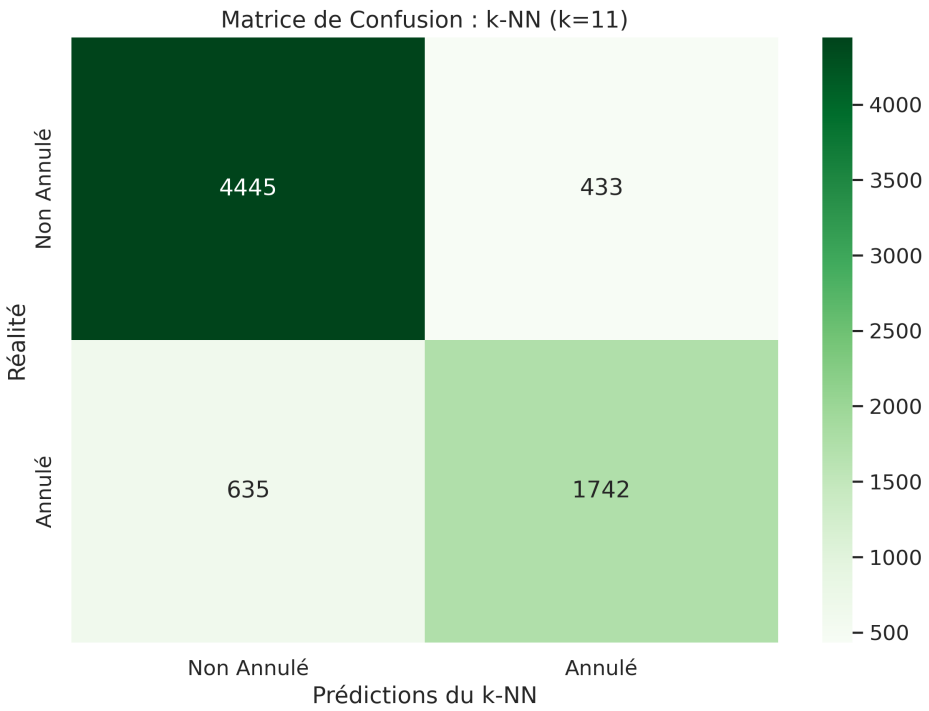

FIGURE 5 – Matrice confusion knn

On peut remarquer que le modèle KNN fait très peu d'erreurs et arrive à prédire assez bien. Vous me direz pourquoi ne pas s'arrêter là ? Le KNN peut être très lourd lorsque il y a beaucoup de données. Il y a aussi la partie métier vu que l'on ne peut pas savoir quelle variable contribue le plus à l'annulation (cela serait intéressant pour la chaîne hôtelière pour pouvoir voir sur quel axe doit-elle travailler pour faire en sorte que ses clients n'annulent pas).

Pour des raisons de compréhension des variables et de vitesse de prédiction, nous essayerons deux autres modèles (à base d'arbre de décision mais qui fonctionnent de deux façons différentes), à savoir le Random Forest et le XGBoost.

3.5 Performance pure : arbre de décision

3.5 Random Forest

Moad

Le Random Forest (300 arbres, `max_depth`=15, `class_weight='balanced'`) atteint un **AUC-ROC de 0,9536** et un **Recall de 0,84** (Precision = 0,84, F1 = 0,84, AUC-PR = 0,9241). C'est un bond significatif par rapport à la régression logistique (+0,08 d'AUC). La cross-validation confirme : AUC-ROC CV = 0,9478 (± 0,003).

Les feature importances confirment nos résultats du Lasso : il semble que `lead_time` domine, suivi de près par `is_long_lead`, `no_of_special_requests` et `avg_price_per_room`. Le gain par rapport aux modèles linéaires prouve que des interactions non linéaires existent dans les données et que les arbres captent naturellement.

Avec un threshold tuning à 0,358, le Recall monte à **0,90** (Precision = 0,74) ce qui permet de détecter 9 annulations sur 10.

3.5 XG BOOST

Brahim

Nous avons atteint un nouveau palier en terme de performance. En effet, les modèles à base d'arbre de décision sont nouvement les meilleurs en terme de prédiction et permettent aussi de comprendre quelles

sont les variables qui contribuent le plus.

Vous me direz pourquoi ajouter encore une autre étape, disons que ce dernier modèle que nous allons implémenter (XGBoost) pourrait atteindre de nouveaux paliers en termes de performances.

En effet, contrairement au Random Forest, qui construit des modèles en parallèle, le XGBoost, lui, les construit de manière séquentielle ; il part du dernier modèle construit et tente de l'améliorer en réduisant le bruit.

C'est ce que nous allons tester maintenant.

Résultat :

voir la figure 16

Comme on s'attendait, le meilleur des modèles en terme de performance, avec AUC-ROC : 0.9600 et un AUC-PR : 0.9347. Cela signifie que le modèle XGBoost est extrêmement précis. Même en baissant le seuil pour attraper plus d'annulations, il fera très peu de fausses alertes par rapport aux autres modèles.

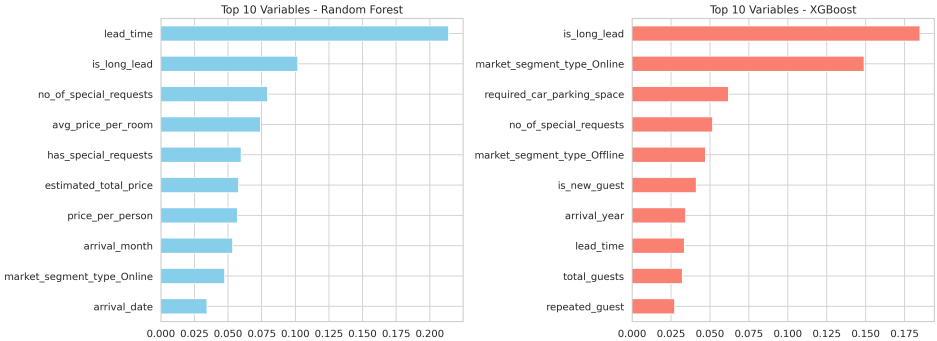

FIGURE 6 – Comparaison importance des variables : XGBoost vs Random Forest.

Ce graphique est assez intéressant, en effet on retrouve certaines variables importantes communes aux deux modèles (mais avec des contributions différentes). La différence majeure est que le Random Forest favorise les variables continues liées au comportement et prix. Alors que le XG BOOST a la capacité de très bien interpréter les variables catégorielles, les interactions et les seuils, ce qui apporte plus d'informations. C'est pour cela que is_long_lead et market_segment_type_Online sont les variables les plus importantes avec le XG BOOST.

3.5 Synthèse comparative

Moad

Modèle	Recall	Precision	F1	AUC-ROC	AUC-PR	Recall CV
LR (base)	0,79	0,65	0,71	0,8722	0,7760	0,776 ± 0,007
LR Lasso	0,82	0,62	0,71	0,8623	0,7567	0,822 ± 0,008
LR Ridge	0,80	0,63	0,70	0,8653	0,7616	0,801 ± 0,006
LDA	0,62	0,76	0,68	0,8683	0,7714	—
QDA	0,81	0,59	0,68	0,8354	0,6916	—
KNN (k=11)	0,73	0,80	0,77	0,9112	0,8393	—
Random Forest	0,84	0,84	0,84	0,9536	0,9241	0,825 ± 0,008
XGBoost	**0,86**	**0,81**	**0,84**	**0,9558**	**0,9265**	**0,853 ± 0,007**

TABLE 1 – Comparaison des modèles sur le jeu de test (seuil = 0,5).

Le XGBoost domine sur l'ensemble des indicateurs. Les écarts-types faibles en cross-validation confirment la stabilité de tous nos modèles. Il sera retenu comme modèle de référence pour l'analyse business.

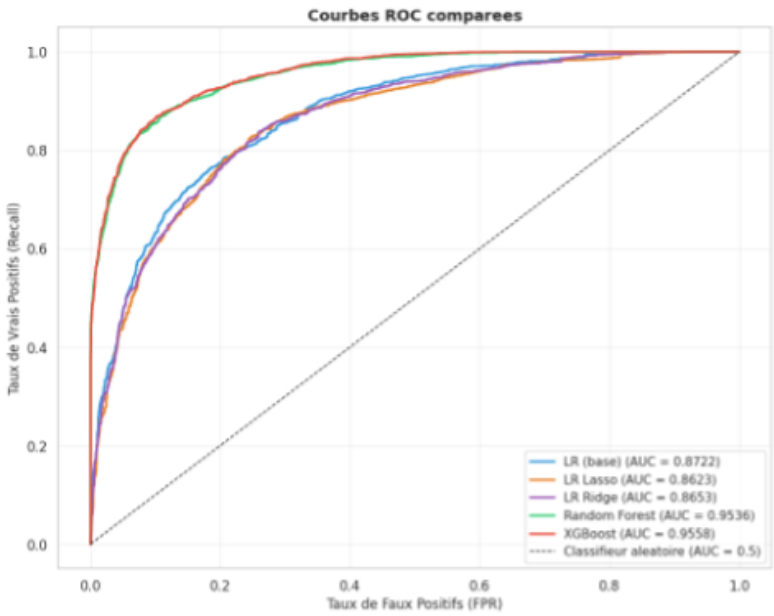

FIGURE 7 – Courbes ROC comparées de tous les modèles.

.

3.5 Analyse business : impact financier

Moad

Ici on va traduire les erreurs du XGBoost en euros. Un faux négatif coûte le revenu total de la réservation (`avg_price_per_room` × `total_nights`, moyenne = 312,81 €). Alors qu'un faux positif coûte environ 30 % du prix (du à la compensation du surbooking, en moyenne = 93,84 €). On va simuler 8 seuils de décision de 0,20 à 0,70.

Au vu des résultats, le seuil optimal est **0,35** avec un **gain net maximal de 688 578 €** sur le jeu de test (Recall = 91,8 %). Avec ce seuil, l'hôtel va anticiper 2 181 annulations et générer 846 fausses alertes, ce qui est acceptable.

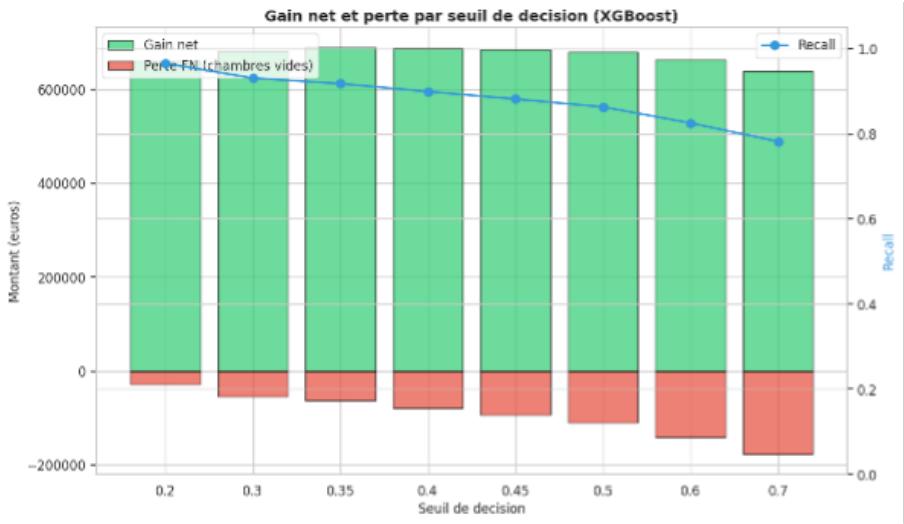

FIGURE 8 – Gain net et perte par seuil de décision (XGBoost).

Profils à risque.

Ce sont les client qui annulent en moyenne avec un lead time de 147 jours, un prix de 110 €/nuit, 76 % n'ont fait aucune demande spéciale et 65 % ont réservé à plus de 90 jours.

Le client fiable quant à lui a un lead time de 55 jours, un prix de 98 €/nuit, et 56 % d'entre eux ont

formulé au moins une special request.

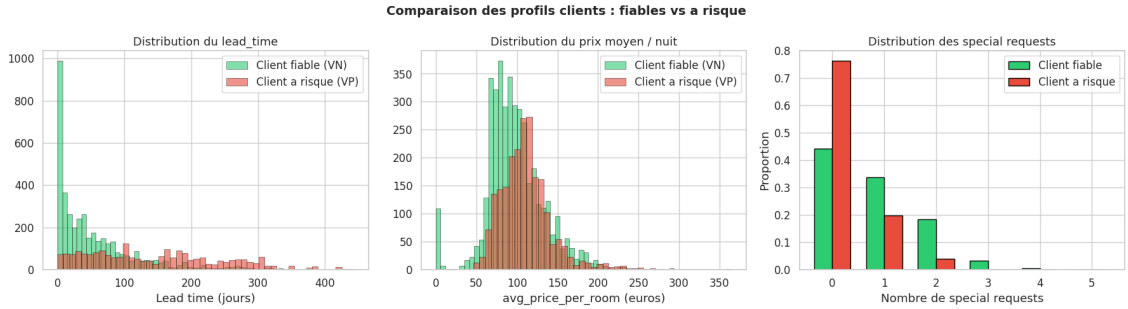

FIGURE 9 – Comparaison des profils : clients fiables (VN) vs clients à risque d'annulation (VP).

Les recommandations qu'on peut faire pour l'hotel.

1. Faire des relances automatisée pour les réservations à `lead_time` > 90 jours (confirmation à J-30, rappel à J-7).
2. Faire des acomptes non remboursables pour les profils à haut risque.
3. Mettre en place un surbooking calibré sur les prédictions du modèle.
4. Encourager les demandes spéciales dès la réservation.

4. Q3 — Identification de profils de réservations par clustering GMM

Contribution : Alexis Mattei

4.1 Question et motivation

La question que je me suis proposé d'étudier est la suivante : peut-on identifier des profils naturels et homogènes de réservations sans utiliser la variable cible, et ces profils sont-ils révélateurs d'un risque d'annulation différencié ?

Ce qui m'a motivé à choisir cette approche, c'est qu'elle est complémentaire à ce que font Moad et Brahim. Plutôt que de prédire directement si une réservation sera annulée, je voulais comprendre *quels types de clients* l'hôtel accueille. Un client fidèle qui réserve tardivement ne représente pas le même risque qu'un nouveau client ayant réservé six mois à l'avance — et ces deux profils méritent des traitements distincts du point de vue de la politique commerciale de l'hôtel.

Au niveau de la méthodologie, j'ai choisi de mobiliser les **modèles de mélanges gaussiens** (GMM) vus en cours, avec l'algorithme EM pour l'estimation et les critères BIC/AIC pour la sélection du nombre de composantes. C'est une approche qui est donc entièrement **non supervisée** puisque je n'utilise à aucun moment la variable cible `booking_status`.

4.2 Méthodologie

Réduction dimensionnelle préalable.

On a vu que le preprocessing commun produit 27 variables standardisées. J'ai choisi d'appliquer une **ACP préalable** avant d'ajuster le GMM, pour deux raisons. D'abord, en grande dimension, les matrices de covariance deviennent difficiles à estimer et l'algorithme EM peut diverger. Ensuite, plusieurs variables sont corrélées entre elles (comme c'est le cas avec `total_nights` et `no_of_week_nights`), ce qui peut biaiser l'estimation des composantes. J'ai donc retenu **10 composantes principales**, qui expliquent **58,7%** de la variance totale — ce qui m'a semblé être un compromis raisonnable entre réduction et préservation de l'information.

Modèle de mélange gaussien.

Un GMM modélise la densité des données comme une somme pondérée de K distributions gaussiennes :

$$f_\beta(x) = \sum_{g=1}^{K} \pi_g \mathcal{N}(x \mid \mu_g, \Sigma_g)$$

où π_g est le poids de la composante g, et μ_g, Σ_g son centre et sa matrice de covariance. Les paramètres sont estimés par **maximum de vraisemblance via l'algorithme EM**, qui alterne d'ailleurs entre le calcul des probabilités d'appartenance *a posteriori* et la mise à jour des paramètres. L'assignation finale de chaque observation à un cluster suit la **règle MAP** :

$$\hat{g}(x) = \arg\max_{1 \leq g \leq K} \pi_g \, \mathcal{N}(x \mid \mu_g, \Sigma_g)$$

Sélection du nombre de clusters.

Pour choisir K, j'ai minimisé le **BIC** (Bayesian Information Criterion), qui pénalise la log-vraisemblance par la complexité du modèle :

$$\text{BIC} = -2\,\ell(\hat{\beta}) + d_K \log(n)$$

J'ai ensuite testé K de 2 à 8, et on observe sur la figure 10 un **coude marqué à** $K = 4$, ce qui montre que le gain marginal en log-vraisemblance devient nettement plus faible au-delà, tandis que l'interprétabilité des clusters se dégrade. J'ai donc retenu $K = 4$. L'algorithme EM converge en **24 itérations**, avec une log-vraisemblance finale de $-7{,}29$.

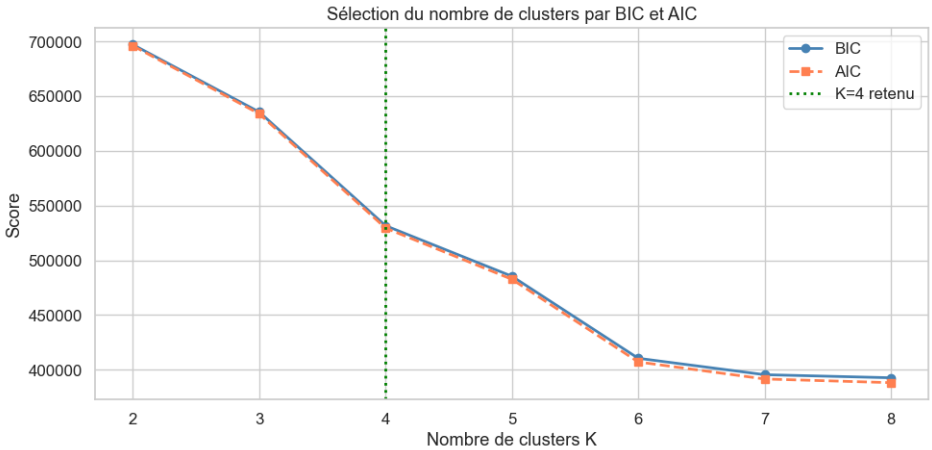

FIGURE 10 – Évolution du BIC et de l'AIC en fonction du nombre de clusters K. Le coude à $K = 4$ justifie le choix retenu.

4.3 Résultats

Visualisation des clusters.

La figure **??** (Annexe **??**) présente les 4 clusters projetés dans le plan des deux premières composantes principales, avec en regard la répartition réelle des annulations. On remarque que le Cluster 3 (orange) se distingue nettement des trois autres dans la partie supérieure gauche de l'espace, ce qui suggère d'emblée un profil structurellement différent.

Profils des clusters.

La figure 11 permet de synthétiser les caractéristiques moyennes de chaque cluster. On voit ainsi quatre profils clairement interprétables :

— **Cluster 0 — Standard** (31,5%, $n = 11\,415$) : ce sont les réservations classiques avec un lead time modéré (79 jours), un prix standard (98€/nuit) et un taux d'annulation de 33,2%. C'est le profil "classique".

— **Cluster 1 — Haut de gamme** (34,7%, $n = 12\,593$) : ce sont les chambres les plus chères (122€/nuit), avec un fort nombre de requêtes spéciales (0,89 en moyenne). On constate que ce cluster affiche pourtant le **taux d'annulation le plus élevé** (36,2%) — ces clients exigeants sont aussi les moins fiables.

— **Cluster 2 — Planificateurs** (25,7%, $n = 9\,309$) : on a un lead time très long (**131 jours**), des prix bas (92€/nuit) et une quasi-absence de requêtes spéciales (0,18). On peut interpréter ce profil comme des réservations opportunistes effectuées très en avance sans engagement fort, ce que confirme un taux d'annulation de 32,7%.

— **Cluster 3 — Fidèles** (8,2%, $n = 2\,958$) : à l'inverse on a ici un lead time court (48 jours) et surtout **31,4% de clients récurrents**. On observe d'ailleurs que ce cluster affiche de loin le **taux d'annulation le plus bas : 16,8%**.

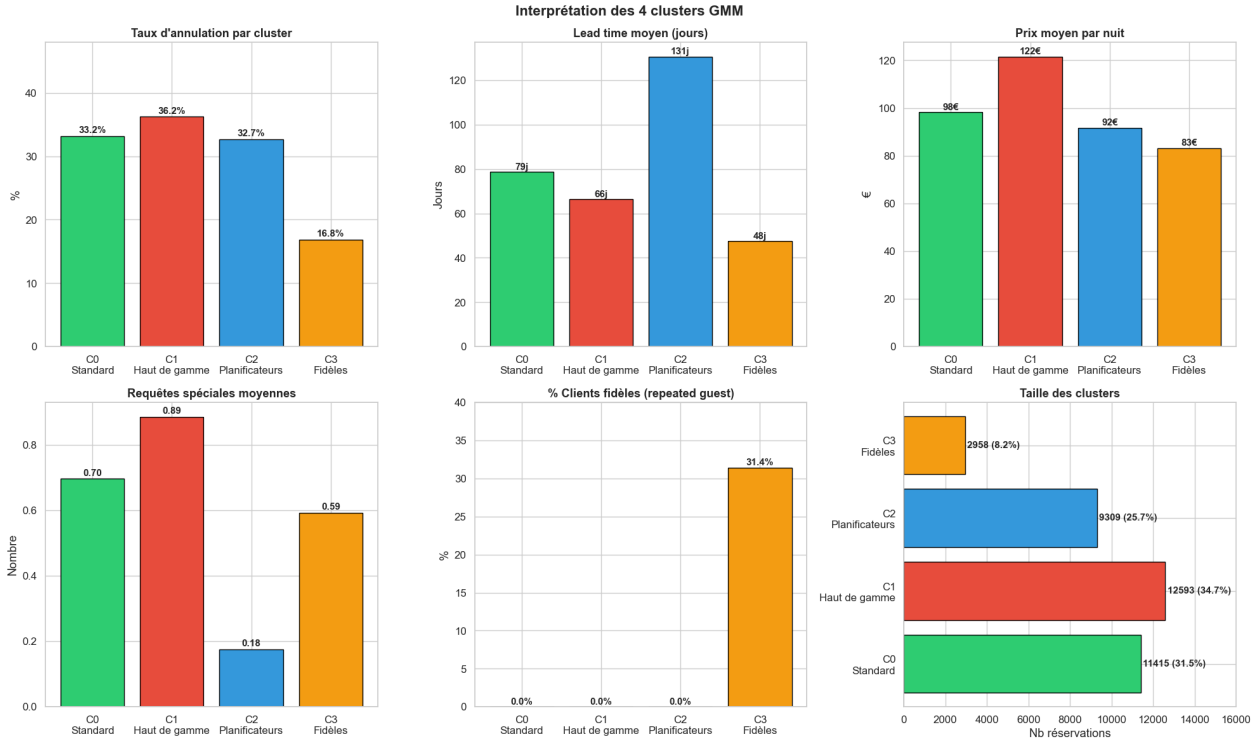

FIGURE 11 – Profils des 4 clusters GMM : taux d'annulation, lead time, prix, requêtes spéciales, fidélité et taille.

Qualité du clustering.

Par rapport au K-Means, l'avantage du GMM est de fournir des **probabilités d'appartenance** plutôt qu'une assignation binaire. J'ai donc analysé ces probabilités (Annexe **??**) pour évaluer la confiance du modèle. On constate que **96,9% des observations ont une probabilité supérieure à 0,9**, et que seulement 0,4% se trouvent en zone d'ambiguïté. Cela confirme ainsi que les 4 clusters sont bien séparés dans l'espace des données.

4.4 Conclusion

Le clustering GMM révèle une structure claire et interprétable dans les données de réservation. Les quatre profils que j'ai identifiés émergent naturellement des données, sans aucune utilisation de la variable cible, et présentent pourtant des taux d'annulation très différenciés, allant de 17% à 36%.

Le résultat qui me semble le plus fort est le fait d'avoir pu identifier le **Cluster 3 (Fidèles)** comme profil à risque quasi-nul. En effet, on voit clairement qu'un lead time court combiné à une fidélité élevée constitue un signal bien plus fiable que le prix ou le standing de la réservation. À l'inverse, on remarque que le **Cluster 1 (Haut de gamme)** cumule prix élevés et taux d'annulation maximum — ce qui suggère que la valeur faciale d'une réservation n'est pas un bon indicateur de sa fiabilité.

A. Annexe- Modèlisation

A.1 LDA et QDA

A.1 LDA

```
============================================================
Linear Discriminant Analysis
============================================================

              precision    recall  f1-score   support

Not Canceled       0.83      0.90      0.87      4878
    Canceled       0.76      0.62      0.68      2377

    accuracy                           0.81      7255
   macro avg       0.79      0.76      0.78      7255
weighted avg       0.81      0.81      0.81      7255


AUC-ROC : 0.8683
AUC-PR : 0.7714
```

Figure 12 – LDA

A.1 QDA

```
============================================================
Quadratic Discriminant Analysis
============================================================

              precision    recall  f1-score   support

Not Canceled       0.89      0.73      0.80      4878
    Canceled       0.59      0.81      0.68      2377

    accuracy                           0.75      7255
   macro avg       0.74      0.77      0.74      7255
weighted avg       0.79      0.75      0.76      7255


AUC-ROC : 0.8354
AUC-PR : 0.6916
```

Figure 13 – QDA

A.2 KNN

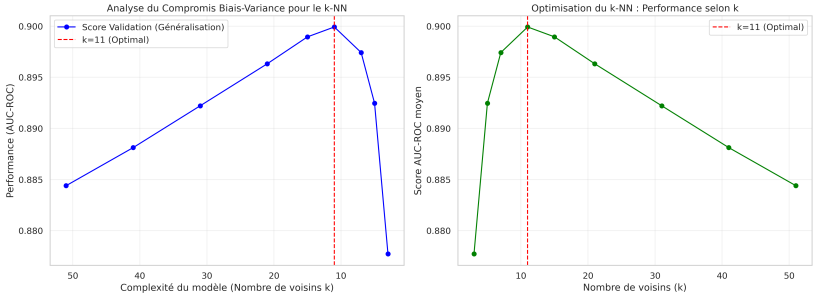

FIGURE 14 – Optimisation du k (nombre de voisins).

```
============================================================
KNN
============================================================

              precision    recall  f1-score   support

Not Canceled       0.88      0.91      0.89      4878
    Canceled       0.80      0.73      0.77      2377

    accuracy                           0.85      7255
   macro avg       0.84      0.82      0.83      7255
weighted avg       0.85      0.85      0.85      7255


AUC-ROC : 0.9112
AUC-PR : 0.8393
```

FIGURE 15 – KNN

A.3 XG BOOST

```
============================================================
XG BOOST
============================================================

              precision    recall  f1-score   support

Not Canceled       0.91      0.94      0.93      4878
    Canceled       0.88      0.82      0.85      2377

    accuracy                           0.90      7255
   macro avg       0.90      0.88      0.89      7255
weighted avg       0.90      0.90      0.90      7255


AUC-ROC : 0.9600
AUC-PR : 0.9347
```

FIGURE 16 – XG BOOST

Annexe — EDA

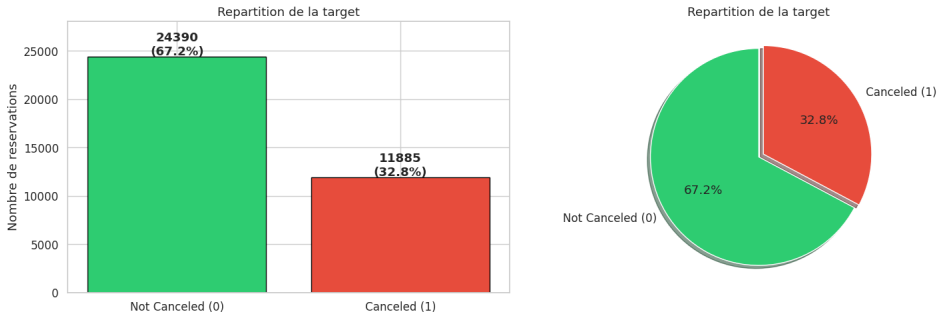

FIGURE 17 – Répartition de la variable cible : 67 % de réservations honorées contre 33 % d'annulations.

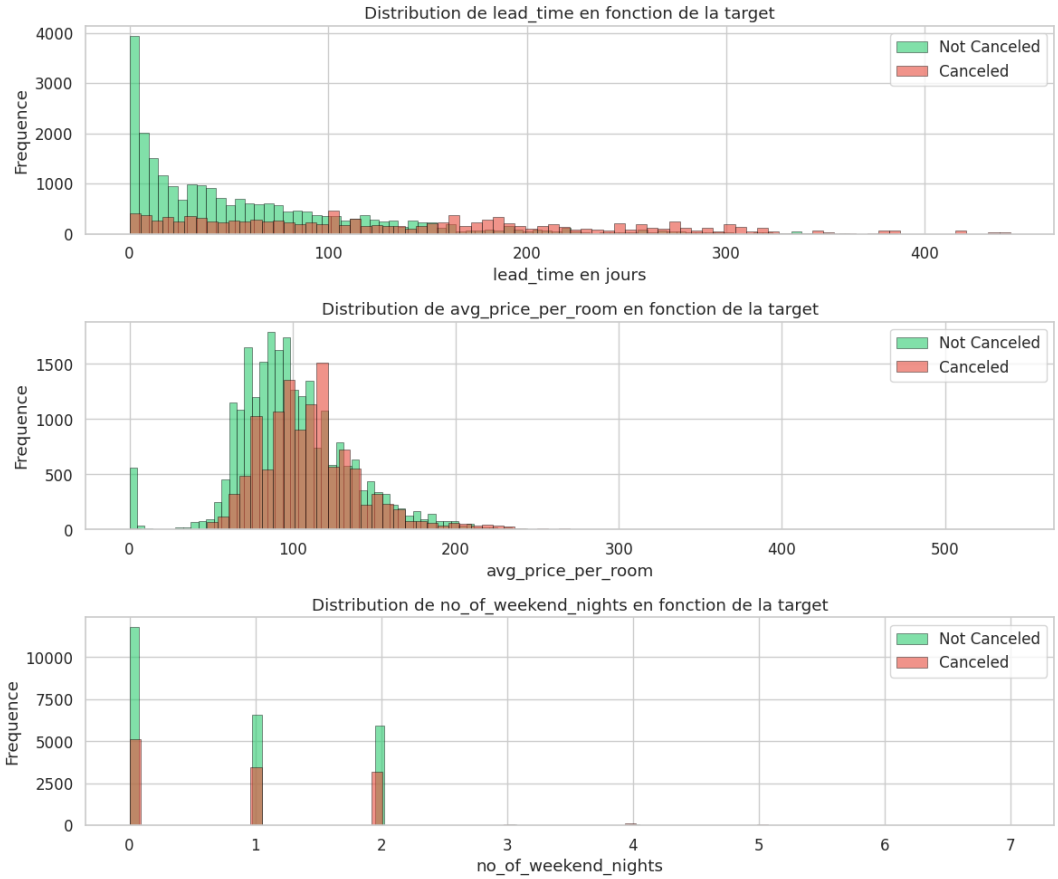

FIGURE 18 – Distributions de `lead_time`, `avg_price_per_room` et `no_of_weekend_nights` par statut d'annulation.

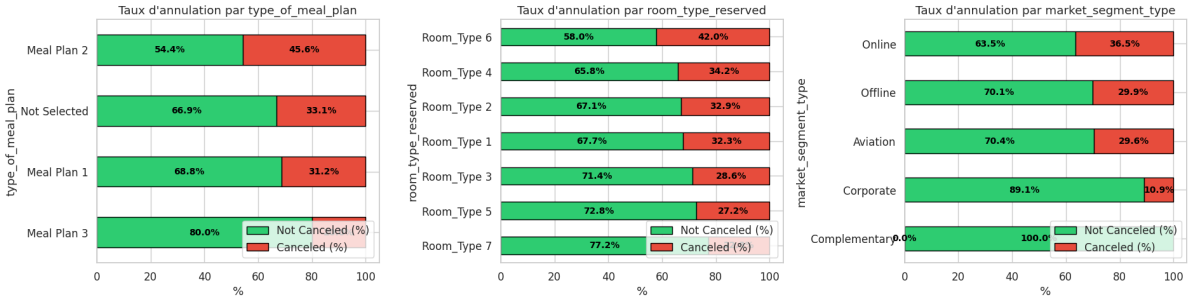

FIGURE 19 – Taux d'annulation par modalité des variables catégorielles.

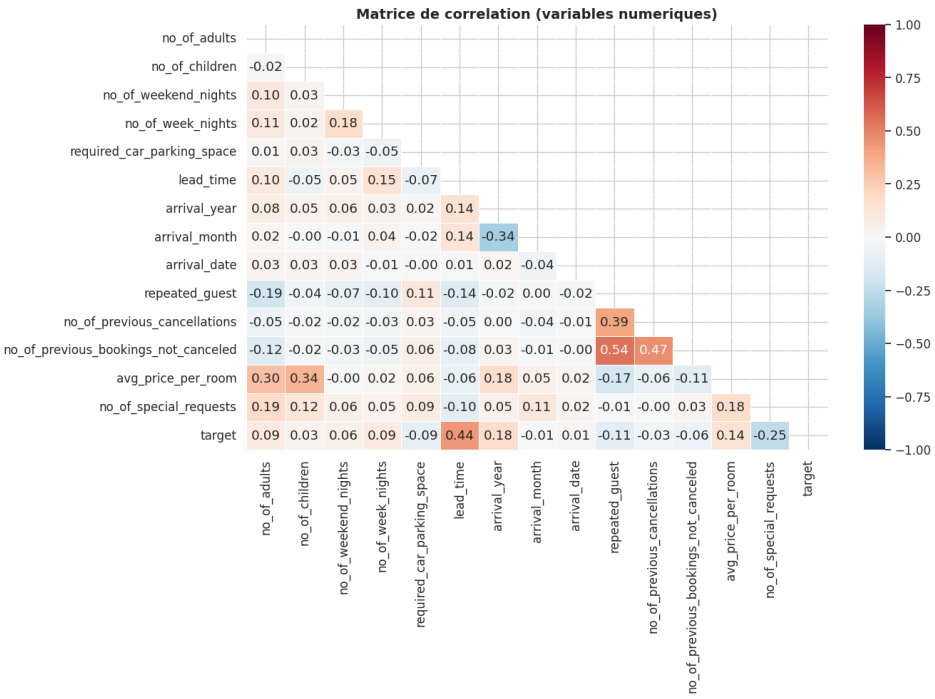

FIGURE 20 – Matrice de corrélation des variables numériques avec la cible.

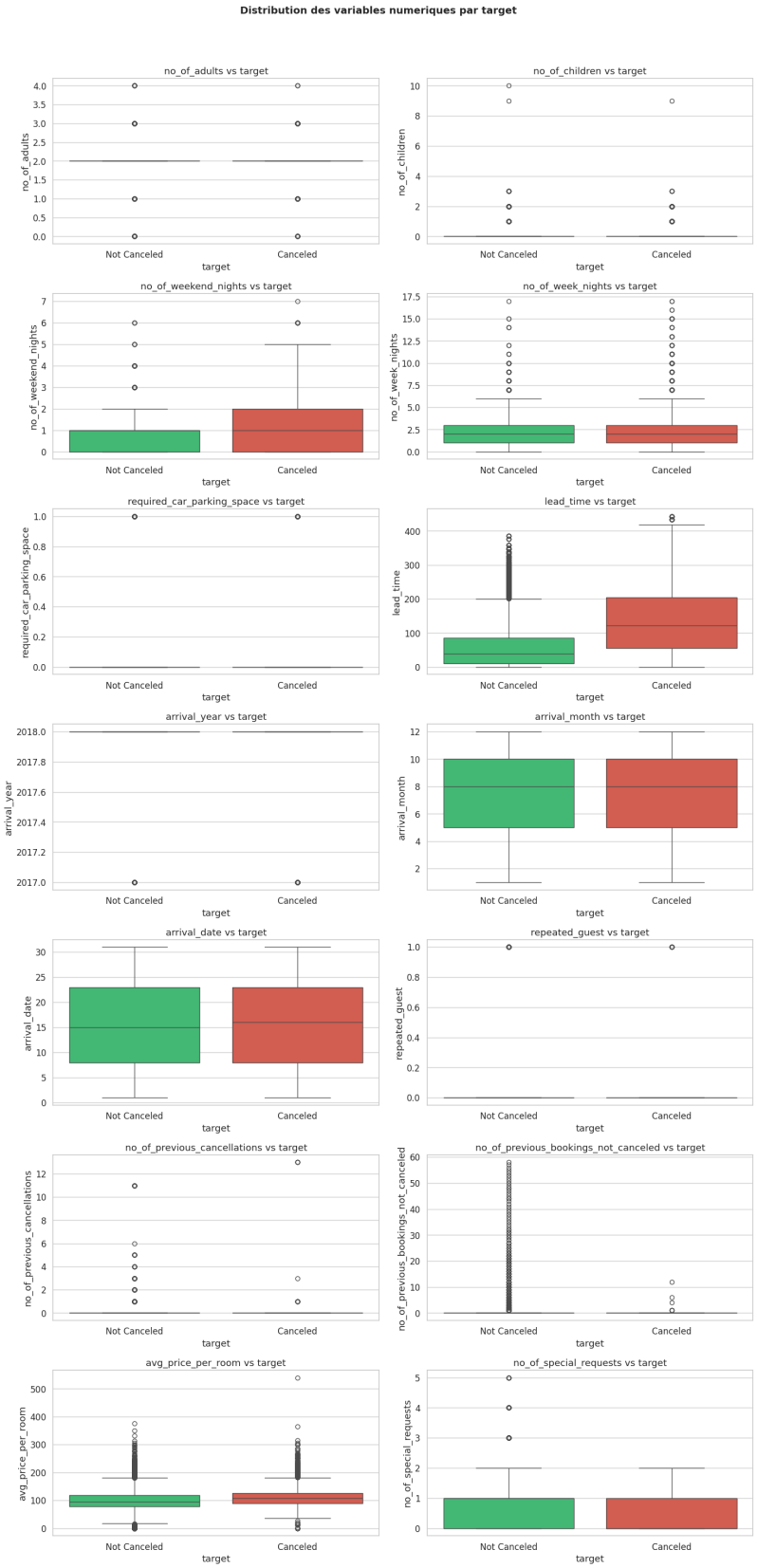

FIGURE 21 – Boxplots de l'ensemble des variables numériques par statut d'annulation.

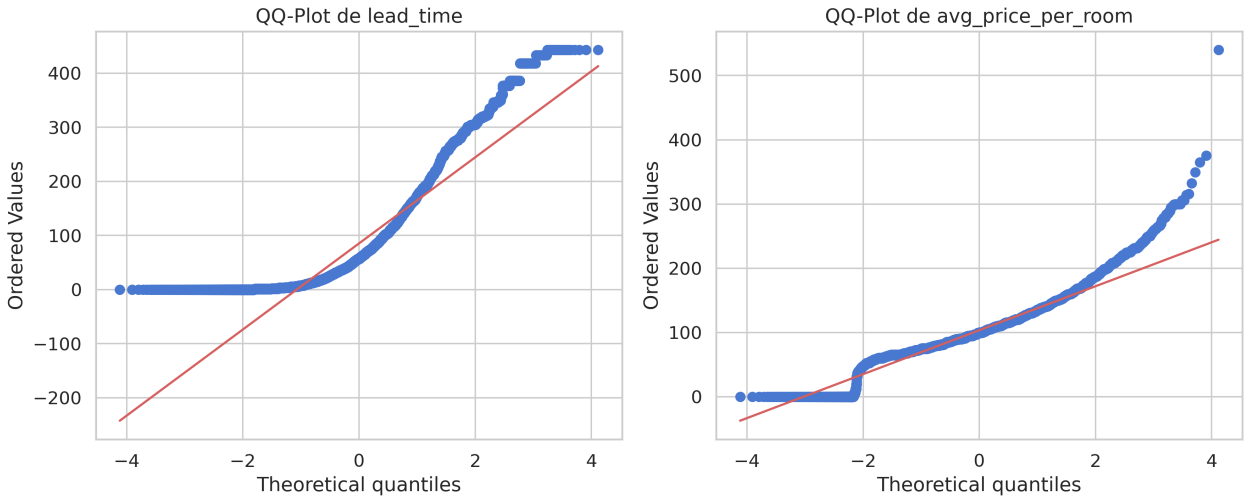

FIGURE 22 – Vérification de le normalité.

Code Python — Q3

Preprocessing

```
X_scaled = pd.DataFrame(scaler.fit_transform(X), columns=X.columns)
print(f"Dimensions : {X_scaled.shape[0]} lignes x {X_scaled.shape[1]} colonnes")
print("Preprocessing termine")
```

```
Dimensions : 36275 lignes x 27 colonnes
Preprocessing terminé
```

ACP préalable

```
pca_pre = PCA(n_components=10, random_state=RANDOM_STATE)
X_pca = pca_pre.fit_transform(X_scaled)
print(f"Variance expliquee par 10 composantes : "
      f"{pca_pre.explained_variance_ratio_.sum():.2%}")
```

```
Variance expliquée par 10 composantes : 58.67%
```

Sélection de K via BIC/AIC

```
bic_scores, aic_scores = [], []
for n in n_components_range:
    gmm = GaussianMixture(n_components=n, covariance_type='full',
                          random_state=RANDOM_STATE, max_iter=200)
    gmm.fit(X_pca)
    bic_scores.append(gmm.bic(X_pca))
    aic_scores.append(gmm.aic(X_pca))
print(f"K retenu : {K_optimal} (coude BIC - bon compromis fit/complexite)")
```

```
K retenu : 4 (coude BIC - bon compromis fit/complexité)
```

GMM final

```
gmm_final = GaussianMixture(n_components=K_optimal, covariance_type='full',
                            random_state=RANDOM_STATE, max_iter=200)
gmm_final.fit(X_pca)
labels = gmm_final.predict(X_pca)
proba  = gmm_final.predict_proba(X_pca)
print(f"Log-vraisemblance : {gmm_final.lower_bound_:.2f}")
print(f"Nombre d'iterations EM : {gmm_final.n_iter_}")
unique, counts = np.unique(labels, return_counts=True)
for k, c in zip(unique, counts):
    print(f"  Cluster {k} : {c} reservations ({c/len(labels)*100:.1f}%)")
```

```
Log-vraisemblance : -7.29
Nombre d'itérations EM : 24
  Cluster 0 :  11415 réservations (31.5%)
  Cluster 1 :  12593 réservations (34.7%)
  Cluster 2 :   9309 réservations (25.7%)
  Cluster 3 :   2958 réservations ( 8.2%)
```

Profil moyen par cluster

```
profile = df_profile.groupby('cluster')[key_vars + ['target']].mean().round(2)
profile['taux_annulation'] = (profile['target'] * 100).round(1)
profile = profile.drop(columns='target')
print(profile.to_string())
```

```
         lead_time  avg_price_per_room  no_of_special_requests
cluster
0            78.67               98.27                    0.70
1            66.42              121.54                    0.89
2           130.70               91.65                    0.18
3            47.58               83.23                    0.59

         repeated_guest  taux_annulation
cluster
0                  0.00             33.0
1                  0.00             36.0
2                  0.00             33.0
3                  0.31             17.0
```

Probabilités d'appartenance

```
max_proba = proba.max(axis=1)
print(f"Probabilite d'appartenance moyenne : {max_proba.mean():.3f}")
print(f"% obs. avec proba > 0.9 : {(max_proba > 0.9).mean()*100:.1f}%")
print(f"% obs. avec proba < 0.6 : {(max_proba < 0.6).mean()*100:.1f}%")
```

```
Probabilité d'appartenance moyenne : 0.989
% observations avec proba > 0.9 : 96.9%
% observations avec proba < 0.6 : 0.4%
```